



# Cyberscope

A *TAC Security* Company

## Audit Report

# Alien Token

June 2026

Network    BSC

Address    0x0eB1c1Aa4C9056f513149324616570D845A9dfb9

Audited by    © cyberscope

# Table of Contents

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>Table of Contents</b>                         | <b>1</b>  |
| <b>Analysis</b>                                  | <b>2</b>  |
| <b>Diagnostics</b>                               | <b>3</b>  |
| <b>Risk Classification</b>                       | <b>4</b>  |
| <b>Review</b>                                    | <b>5</b>  |
| Audit Updates                                    | 5         |
| Source Files                                     | 5         |
| <b>Findings Breakdown</b>                        | <b>6</b>  |
| L04 - Conformance to Solidity Naming Conventions | 7         |
| Description                                      | 7         |
| Recommendation                                   | 8         |
| L18 - Multiple Pragma Directives                 | 9         |
| Description                                      | 9         |
| Recommendation                                   | 9         |
| L19 - Stable Compiler Version                    | 10        |
| Description                                      | 10        |
| Recommendation                                   | 10        |
| L20 - Succeeded Transfer Check                   | 11        |
| Description                                      | 11        |
| Recommendation                                   | 11        |
| <b>Functions Analysis</b>                        | <b>12</b> |
| <b>Inheritance Graph</b>                         | <b>13</b> |
| <b>Flow Graph</b>                                | <b>14</b> |
| <b>Summary</b>                                   | <b>15</b> |
| <b>Disclaimer</b>                                | <b>16</b> |
| <b>About Cyberscope</b>                          | <b>17</b> |

# Analysis

● Critical   ● Medium   ● Minor / Informative   ● Pass

| Severity | Code | Description             | Status |
|----------|------|-------------------------|--------|
| ●        | ST   | Stops Transactions      | Passed |
| ●        | OTUT | Transfers User's Tokens | Passed |
| ●        | ELFM | Exceeds Fees Limit      | Passed |
| ●        | MT   | Mints Tokens            | Passed |
| ●        | BT   | Burns Tokens            | Passed |
| ●        | BC   | Blacklists Addresses    | Passed |

# Diagnostics

● Critical ● Medium ● Minor / Informative

| Severity | Code | Description                                | Status     |
|----------|------|--------------------------------------------|------------|
| ●        | L04  | Conformance to Solidity Naming Conventions | Unresolved |
| ●        | L18  | Multiple Pragma Directives                 | Unresolved |
| ●        | L19  | Stable Compiler Version                    | Unresolved |
| ●        | L20  | Succeeded Transfer Check                   | Unresolved |

## Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

| Severity              | Likelihood / Impact of Exploitation                      |
|-----------------------|----------------------------------------------------------|
| ● Critical            | Highly Likely / High Impact                              |
| ● Medium              | Less Likely / High Impact or Highly Likely/ Lower Impact |
| ● Minor / Informative | Unlikely / Low to no Impact                              |

## Review

|                         |                                                                                                                                                             |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Contract Name</b>    | Alien                                                                                                                                                       |
| <b>Compiler Version</b> | v0.8.24+commit.e11b9ed9                                                                                                                                     |
| <b>Optimization</b>     | 200 runs                                                                                                                                                    |
| <b>Explorer</b>         | <a href="https://bscscan.com/address/0x0eb1c1aa4c9056f513149324616570d845a9dfb9">https://bscscan.com/address/0x0eb1c1aa4c9056f513149324616570d845a9dfb9</a> |
| <b>Address</b>          | 0x0eb1c1aa4c9056f513149324616570d845a9dfb9                                                                                                                  |
| <b>Network</b>          | BSC                                                                                                                                                         |
| <b>Symbol</b>           | Alien                                                                                                                                                       |
| <b>Decimals</b>         | 18                                                                                                                                                          |
| <b>Total Supply</b>     | 10,192,000,000,000,000                                                                                                                                      |

## Audit Updates

|                      |              |
|----------------------|--------------|
| <b>Initial Audit</b> | 15 June 2026 |
|----------------------|--------------|

## Source Files

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| <b>Filename</b>  | SHA256                                                           |
| <b>Alien.sol</b> | 7a853c6303879b7da6b31092dd0faae64551d672ac88da09ab7a77193ea52103 |

## Findings Breakdown



|                       |   |
|-----------------------|---|
| ● Critical            | 0 |
| ● Medium              | 0 |
| ● Minor / Informative | 4 |

| Severity              | Unresolved | Acknowledged | Resolved | Other |
|-----------------------|------------|--------------|----------|-------|
| ● Critical            | 0          | 0            | 0        | 0     |
| ● Medium              | 0          | 0            | 0        | 0     |
| ● Minor / Informative | 4          | 0            | 0        | 0     |

## L04 - Conformance to Solidity Naming Conventions

|                    |                     |
|--------------------|---------------------|
| <b>Criticality</b> | Minor / Informative |
| <b>Location</b>    | Alien.sol#L729,730  |
| <b>Status</b>      | Unresolved          |

### Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX\_VALUE, ERROR\_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

Shell

```
private NAME;  
private SYMBOL;
```



## Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

## L18 - Multiple Pragma Directives

|                    |                                     |
|--------------------|-------------------------------------|
| <b>Criticality</b> | Minor / Informative                 |
| <b>Location</b>    | Alien.sol#L6,88,116,146,311,618,718 |
| <b>Status</b>      | Unresolved                          |

### Description

If the contract includes multiple conflicting pragma directives, it may produce unexpected errors. To avoid this, it's important to include the correct pragma directive at the top of the contract and to ensure that it is the only pragma directive included in the contract.

Shell

```
pragma solidity >=0.4.16;  
pragma solidity >=0.6.2;  
pragma solidity ^0.8.20;  
pragma solidity ^0.8.20;  
pragma solidity >=0.8.4;
```

### Recommendation

It is important to include only one pragma directive at the top of the contract and to ensure that it accurately reflects the version of Solidity that the contract is written in.

By including all required compiler options and flags in a single pragma directive, the potential conflicts could be avoided and ensure that the contract can be compiled correctly.

## L19 - Stable Compiler Version

|                    |                     |
|--------------------|---------------------|
| <b>Criticality</b> | Minor / Informative |
| <b>Location</b>    | Alien.sol#L311      |
| <b>Status</b>      | Unresolved          |

### Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
Shell  
solidity ^0.8.20;
```

### Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

## L20 - Succeeded Transfer Check

|                    |                     |
|--------------------|---------------------|
| <b>Criticality</b> | Minor / Informative |
| <b>Location</b>    | Alien.sol#L860      |
| <b>Status</b>      | Unresolved          |

### Description

According to the ERC20 specification, the transfer methods should be checked if the result is successful. Otherwise, the contract may wrongly assume that the transfer has been established.

```
Shell
(tokenAddress).transfer(owner(), amount);
```

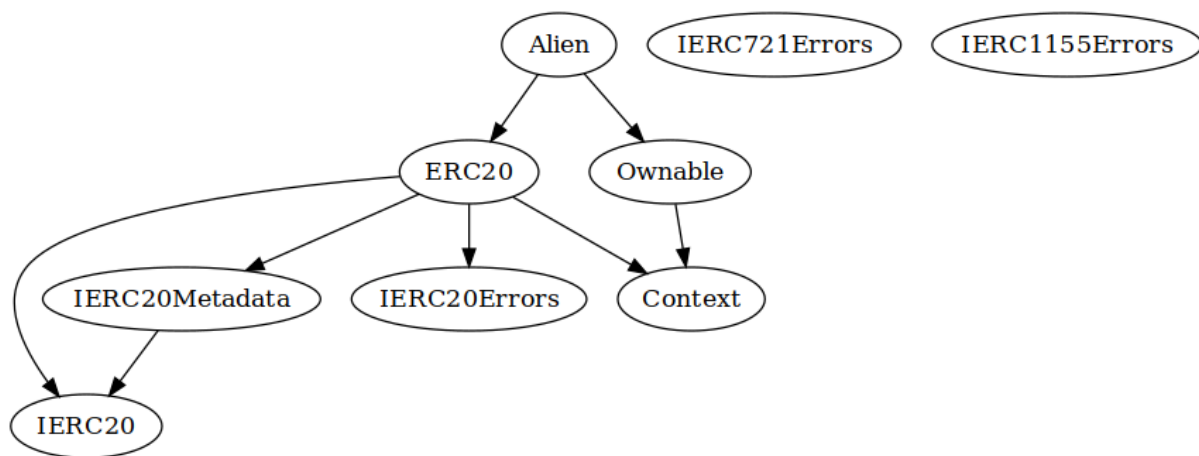
### Recommendation

The contract should check if the result of the transfer methods is successful. The team is advised to check the SafeERC20 library from the [Openzeppelin library](#).

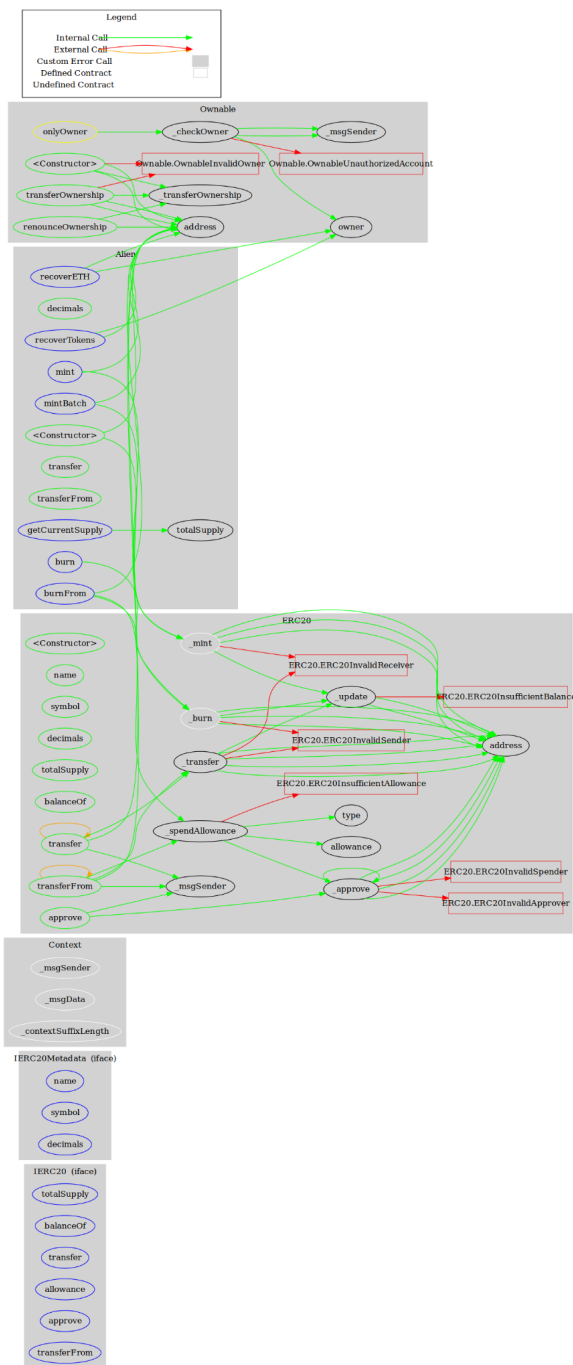
# Functions Analysis

| Contract | Type             | Bases          |            |               |
|----------|------------------|----------------|------------|---------------|
|          | Function Name    | Visibility     | Mutability | Modifiers     |
|          |                  |                |            |               |
| Alien    | Implementation   | ERC20, Ownable |            |               |
|          |                  | Public         | ✓          | ERC20 Ownable |
|          | decimals         | Public         |            | -             |
|          | mint             | External       | ✓          | onlyOwner     |
|          | mintBatch        | External       | ✓          | onlyOwner     |
|          | burn             | External       | ✓          | onlyOwner     |
|          | burnFrom         | External       | ✓          | onlyOwner     |
|          | transfer         | Public         | ✓          | -             |
|          | transferFrom     | Public         | ✓          | -             |
|          | recoverTokens    | External       | ✓          | onlyOwner     |
|          | recoverETH       | External       | ✓          | onlyOwner     |
|          | getCurrentSupply | External       |            | -             |

## Inheritance Graph



# Flow Graph



## Summary

Alien contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. Alien is an interesting project that has a friendly and growing community. The Smart Contract analysis reported no compiler error or critical issues

The contract has renounced the ownership so it no longer has an assigned owner and consequently, the owner's privileges and authority are revoked. As a result, the owner is unable to execute any methods that are designated exclusively for owner access. By relinquishing ownership, the contract eliminates the potential risks associated with centralized authority, reducing the possibility of the owner misusing their privileges or becoming a single point of failure. It is important to note that renouncing ownership is an irreversible action, and once executed, it cannot be undone. The ownership has been renounced on this transaction:

<https://bscscan.com/tx/0x9af13094620914e6365a06fced42889b17043966d3c562b7dad568477f0d9d5>



## Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

# About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

[cyberscope.io](https://cyberscope.io)